

- Document 01: The 9-Node Agentic AI Pipeline
 - CodePerfect Auditor — Technical Code Walkthrough
 - Overview
 - The Shared State Object — agents/graph.py
 - Explanation
 - The Graph Compiler — build_integronix_graph()
 - Explanation
 - NODE 1: Document Processor — agents/doc_processor.py
 - Explanation
 - NODE 2: Clinical Extractor — agents/clinical_extractor.py
 - Explanation
 - NODE 3: SNOMED Resolver — agents/snomed_resolver.py
 - Explanation
 - NODE 4: SNOMED→ICD Mapper — agents/snomed_icd_mapper.py
 - Explanation
 - NODE 5: ICD Embedding Fallback — agents/icd_embedding.py
 - Explanation
 - NODE 6: ICD Decision Engine — agents/icd_decision.py
 - Explanation
 - NODE 7: Audit Comparison — agents/audit_comparison.py
 - Explanation
 - NODE 8: Risk Scorer — agents/risk_scoring.py
 - Explanation
 - NODE 9: Financial Calculator — agents/financial_calculator.py
 - Explanation
 - Pipeline Flow Summary

Document 01: The 9-Node Agentic AI Pipeline

CodePerfect Auditor — Technical Code Walkthrough

Overview

The intelligence of CodePerfect Auditor is driven by a **9-Node Stateful Agentic Pipeline** built using **LangGraph** — a specialized AI orchestration library designed to coordinate multi-step, stateful workflows. Unlike simple LLM chatbots that blindly rely on a single language model call, this pipeline enforces a deterministic, auditable, and multi-source clinical decision process.

The state object **CodingState** acts as the "patient's chart" — a shared TypedDict that travels through every node, getting enriched with data at each stage. No node can bypass another, ensuring full traceability.

The Shared State Object — **agents/graph.py**

Every node reads from and writes to the **CodingState** object. This is the foundational data contract of the entire pipeline.

```
"""
This file defines the heart of our application: the LangGraph pipeline.
It connects all the individual agents (nodes) into a coherent workflow,
managing the flow of data from one step to the next.
"""

from typing import TypedDict, Optional, List

class CodingState(TypedDict, total=False):
    # — Input Data —————
    session_id: str
    org_id: Optional[str]          # Organization ID for pricing multiplier lookup
    icd_version: Optional[str]     # ICD-10 | ICD-11 (from org_settings)
    claim_scheme: Optional[str]   # ayushman | cghs | etc (from org_settings)
    coding_mode: Optional[str]    # aggressive | balanced | conservative
    pdf_bytes: Optional[bytes]
    human_icd_code: Optional[str] # The code entered by a human for comparison

    # — Document metadata (set by Node 1) —————
    document_source: Optional[str] # "text_input" | "pdf_upload"
```

```

ocr_used: Optional[bool]          # True if Tesseract OCR fallback was triggered

# — Pipeline Data —
raw_text: str                     # Output from Node 1
structured_entities: dict         # Output from Node 2 (the LLM)
procedures_and_services: List[str]
resolved_snomed_code: Optional[str]
direct_mapped_icd: Optional[str]
candidate_icd_codes: List[dict]
final_icd_code: str              # Output from Node 6 (the final decision)
confidence_score: float
icd_codes: List[dict]
cpt_codes: List[dict]

# — Audit & Risk Analysis —
discrepancy_type: Optional[str]
financial_delta: Optional[float]
drg_flag: Optional[str]
risk_score: float                # Output from Node 8
risk_label: str

# — Financial Summary —
financial_summary: Optional[dict]

# — Error Handling —
error_at: Optional[str]
error_detail: Optional[str]

```

Explanation

The **CodingState** TypedDict enforces strict Python typing across all nodes. Using **total=False** means no field is mandatory at initialization — each node declares what it produces. This prevents a crashed upstream node from blocking all downstream nodes.

The Graph Compiler — **build_integronix_graph()**

```

def build_integronix_graph():
    from langgraph.graph import StateGraph, END
    from agents.doc_processor import doc_processing_node
    from agents.clinical_extractor import clinical_extraction_agent
    from agents.cpt_resolver import cpt_resolver_node
    from agents.snomed_resolver import snomed_resolver_node
    from agents.snomed_icd_mapper import snomed_icd_mapping_node

```

```

from agents.icd_embedding      import icd_embedding_node
from agents.icd_decision      import icd_decision_node
from agents.audit_comparison  import audit_comparison_node
from agents.risk_scoring      import risk_scoring_node
from agents.financial_calculator import financial_calculator_node

graph = StateGraph(CodingState)

# Register every agent as a named node
graph.add_node("doc_processing", doc_processing_node)
graph.add_node("clinical_extract", clinical_extraction_agent)
graph.add_node("cpt_resolve", cpt_resolver_node)
graph.add_node("snomed_resolve", snomed_resolver_node)
graph.add_node("snomed_icd_map", snomed_icd_mapping_node)
graph.add_node("icd_embedding", icd_embedding_node) # Fallback: vector
search
graph.add_node("icd_decision", icd_decision_node) # Deterministic rule
engine
graph.add_node("audit_comparison", audit_comparison_node)
graph.add_node("risk_scoring", risk_scoring_node)
graph.add_node("financial_calc", financial_calculator_node)

# Wire the execution pipeline
graph.set_entry_point("doc_processing")
graph.add_edge("doc_processing", "clinical_extract")
graph.add_edge("clinical_extract", "cpt_resolve")
graph.add_edge("cpt_resolve", "snomed_resolve")
graph.add_edge("snomed_resolve", "snomed_icd_map")

# CONDITIONAL EDGE: If direct SNOMED->ICD mapping was found, skip embedding.
# If no direct mapping found, trigger the vector similarity fallback.
graph.add_conditional_edges(
    "snomed_icd_map",
    _route_after_mapping,
    {
        "icd_decision": "icd_decision", # Direct path (fast)
        "icd_embedding": "icd_embedding", # Fallback path (semantic search)
    },
)
graph.add_edge("icd_embedding", "icd_decision")
graph.add_edge("icd_decision", "audit_comparison")
graph.add_edge("audit_comparison", "risk_scoring")
graph.add_edge("risk_scoring", "financial_calc")
graph.add_edge("financial_calc", END)

return graph.compile()

```

Explanation

`StateGraph` from `LangGraph` manages the execution order. The key innovation here is the **conditional edge** after `snomed_icd_map`. Rather than always running the expensive `SentenceTransformer` embedding model, this router checks if the previous

node already found a valid ICD candidate. If yes, the pipeline short-circuits directly to the decision engine — drastically improving latency for well-documented clinical cases.

NODE 1: Document Processor —

agents/doc_processor.py

Purpose: Ingest the clinical document (plain text or PDF) and extract raw string content.

```
"""
This is the very first step in our process. This agent takes the initial
document – which can be a PDF or plain text – and gets it ready for the
rest of the pipeline by extracting the raw text content.
"""

from agents.graph import CodingState
from agents.node_runner import safe_node
from services.pdf_service import extract_text_from_pdf
from exceptions import PDFExtractionError

@safe_node("doc_processing")
async def doc_processing_node(state: CodingState) -> CodingState:

    # If text is already present (user pasted text directly), skip PDF parsing.
    if state.get("raw_text"):
        state.setdefault("document_source", "text_input")
        state.setdefault("ocr_used", False)
        return state

    # If a PDF binary is present, extract text from it.
    pdf_bytes = state.get("pdf_bytes")
    if not pdf_bytes:
        raise PDFExtractionError(
            "Neither raw_text nor pdf_bytes found in state. "
            "Set one before invoking the graph."
        )

    # extract_text_from_pdf returns a (text, ocr_used) tuple.
    # If PyMuPDF fails (e.g., scanned/image PDF), it falls back to Tesseract OCR.
    raw_text, ocr_used = extract_text_from_pdf(pdf_bytes)
    state["raw_text"] = raw_text
    state["document_source"] = "pdf_upload"
    state["ocr_used"] = ocr_used
    return state
```

Explanation

Node 1 supports two ingestion paths: direct text and binary PDF upload. The `@safe_node` decorator is a custom wrapper (in `node_runner.py`) that catches any unhandled exception and writes an error record to `state["error_at"]` instead of crashing the entire pipeline. The `ocr_used` flag persists in `CodingState` so that downstream nodes and the final audit log know whether OCR degradation may have affected the extracted text quality.

NODE 2: Clinical Extractor — `agents/clinical_extractor.py`

Purpose: Use a Large Language Model (LLM) to identify structured clinical entities (diagnoses, symptoms, procedures) from raw text.

```
"""
This agent uses a Large Language Model (LLM) to extract clinical entities
from the raw medical text. It's a critical first step in understanding the
patient's condition, pulling out diagnoses, symptoms, and procedures.
"""

from agents.graph import CodingState
from agents.node_runner import safe_node
from services.extraction_service import extract_clinical_entities

@safe_node("clinical_extract")
async def clinical_extraction_agent(state: CodingState) -> CodingState:
    raw_text = state.get("raw_text", "")
    session_id = str(state.get("session_id", ""))

    if not raw_text:
        raise ValueError("raw_text is empty – doc_processing_node must run first")

    # The extraction_service sends the raw text to Groq (llama-3.3-70b) with a
    # structured medical NER prompt and parses the result into a typed Pydantic
    model.
    extraction, metadata = await extract_clinical_entities(raw_text,
        session_id=session_id)

    # structured_entities is a Pydantic model dumped to dict:
    # { "diagnoses": [...], "symptoms": [...], "procedures_and_services": [...] }
    state["structured_entities"] = extraction.model_dump()
    state["procedures_and_services"] = extraction.procedures_and_services
    state["extraction_metadata"] = metadata # model name, token counts,
    latency_ms
    return state
```

Explanation

This node sends the raw clinical text to a hosted LLM (Groq `llama-3.3-70b-versatile`) with a specialized Named Entity Recognition (NER) medical prompt. It returns a strictly validated `Pydantic` model containing a list of diagnosis objects, each containing `text`, `severity`, and crucial `evidence_text` (the exact sentence from the chart that supports the diagnosis). This `evidence_text` field is later used by Node 6's Clinical Consistency Scorer to validate code legitimacy.

NODE 3: SNOMED Resolver — `agents/snomed_resolver.py`

Purpose: Map each extracted diagnosis phrase to an official SNOMED CT clinical concept code using a vector similarity search on locally cached SNOMED embeddings.

Explanation

Node 3 takes each diagnosis text string from `state["structured_entities"]` and converts it into a 384-dimensional mathematical vector using `SentenceTransformer("all-MiniLM-L6-v2")`. It then queries the `snomed_concepts` table in Supabase (which holds `VECTOR(384)` columns) via the `match_snomed_concepts` PostgreSQL RPC function to find the nearest SNOMED concept by cosine similarity. The resolved SNOMED code (e.g. `44054006` for Type 2 Diabetes) is stored in `state["resolved_snomed_code"]` and passed to Node 4.

NODE 4: SNOMED→ICD Mapper — `agents/snomed_icd_mapper.py`

Purpose: Cross-walk the SNOMED CT code to a specific ICD-10 or ICD-11 billing code using the deterministic `snomed_icd_map` crosswalk table.

Explanation

This node queries the `snomed_icd_map` table — a pre-loaded ontology lookup table mapping SNOMED codes to ICD billing codes with a typed `mapping_type` (`exact`, `narrower`, `broader`, or `approximate`). Crucially, only `exact` or `narrower` mappings are trusted without additional scoring. If a valid ICD candidate is found, `state["candidate_icd_codes"]` is populated and the conditional edge routes directly to Node 6, **bypassing the expensive embedding fallback in Node 5 entirely**.

NODE 5: ICD Embedding Fallback — `agents/icd_embedding.py`

Purpose: When no direct SNOMED→ICD mapping exists, perform a direct semantic vector search against the entire ICD code database.

```
SIMILARITY_THRESHOLD = 0.55    # Minimum cosine similarity to accept a match.
EMBEDDING_TOP_K = 5           # Return top 5 most semantically similar codes.

@safe_node("icd_embedding")
async def icd_embedding_node(state: CodingState) -> CodingState:
    entities = state.get("structured_entities") or {}
    diagnoses = entities.get("diagnoses", [])

    # Use the primary (first) diagnosis text as the embedding query
    primary_text = diagnoses[0].get("text", "").strip()
    primary_chapter = _infer_chapter_from_diagnoses(entities)
    excluded_chapters = UNRELATED_CHAPTER_PAIRS.get(primary_chapter, set())

    # Encode the diagnosis text to a 384-dim float vector
    query_vector = _embed_text(primary_text)

    # Query pgvector via Supabase RPC — returns cosine-ranked ICD candidates
    results = await rpc("match_icd_codes", {
        "query_embedding": _vector_to_pg_literal(query_vector),
        "similarity_threshold": SIMILARITY_THRESHOLD,
        "match_count": EMBEDDING_TOP_K + 5,
    })

    # Apply chapter exclusion guardrail to prevent cross-specialty mistakes.
    # e.g. a Respiratory query must never suggest a Mental Health code.
    filtered = [
        r for r in results
        if r.get("chapter") not in excluded_chapters
   ][:EMBEDDING_TOP_K]

    state["candidate_icd_codes"] = candidates
```



```
state["mapping_path"] = "embedding"
return state
```

Explanation

The `UNRELATED_CHAPTER_PAIRS` dictionary acts as a clinical guardrail — preventing the system from ever suggesting a psychiatric code when processing a cardiac chart. The `match_icd_codes` RPC is a PostgreSQL function that uses the HNSW (Hierarchical Navigable Small World) index on the `icd_codes.embedding` column for sub-200ms Approximate Nearest Neighbor search across 71,000+ medical concepts.

NODE 6: ICD Decision Engine — `agents/icd_decision.py`

Purpose: Select the final, optimal ICD code from all candidates using a deterministic, weighted composite scoring algorithm (NO LLM involved — pure clinical logic).

```
def _final_score(candidate: dict, entities: dict, raw_text: str = "") -> float:
    """
    Weighted composite scoring formula:
        confidence (40%) + specificity (30%) + consistency (20%) + combination (10%)
    - negation penalty
    """
    confidence = float(candidate.get("confidence", 0.85))
    specificity = _specificity_score(candidate, entities) # Code length +
    complication keywords
    consistency = _clinical_consistency_score(candidate, entities) # Terminology
    overlap
    combination = _combination_code_priority(candidate) # Prefers "with" codes
    (ICD guidelines)
    negation = _negation_penalty(candidate, entities, raw_text) # Penalises
    contradicted codes

    score = (
        confidence * 0.40 +
        specificity * 0.30 +
        consistency * 0.20 +
        combination * 0.10 +
        negation # Can be negative (penalty)
    )
    return round(max(0.0, min(score, 1.0)), 4)
```

Explanation

This is the most critical node. The decision is NOT made by an LLM — it is made by a transparent, auditable mathematical scoring formula. This is intentional: healthcare regulators require explainable AI that can be justified in a court of law.

Key scoring rules:

- **Specificity:** Longer ICD codes (e.g., **E11.3211**) are preferred over shorter vague codes (e.g., **E11**) because they capture more clinical detail and higher reimbursement.
 - **Negation Penalty:** If the clinical chart says "no complications" but a candidate code implies complications, it receives a -0.4 penalty, preventing overcoding.
 - **Gold Standard Override:** Specific medical abbreviations like **NSTEMI** are directly mapped to **I21.4** with a 0.98 confidence, overriding all scoring logic.
-

NODE 7: Audit Comparison — **agents/audit_comparison.py**

Purpose: Compare the AI-generated code against any human-entered code and classify the discrepancy type for transparency and compliance.

Explanation

If a human coder has entered a code in `state["human_icd_code"]`, this node classifies the outcome into one of five discrepancy types:

- **EXACT_MATCH** — AI agrees with human (no risk)
- **SPECIFICITY_IMPROVEMENT** — AI found a more specific variant of the human's code
- **OVERCODING** — AI's code implies a higher severity than the chart supports
- **CODE_DIVERGENCE** — AI and human chose completely different diagnostic categories
- **UNSUPPORTED_CODE** — The human's code cannot be found in the ontology database

The financial delta (`state["financial_delta"]`) is calculated by subtracting the human code's base reimbursement from the AI code's base reimbursement, directly quantifying the revenue impact of the coding discrepancy.

NODE 8: Risk Scorer — `agents/risk_scoring.py`

Purpose: Calculate a probability-based audit risk score and persist all pipeline results to the PostgreSQL database.

```
DISCREPANCY_RISK = {
    "EXACT_MATCH": 0.0,
    "NO_COMPARISON": 0.1,
    "SPECIFICITY_IMPROVEMENT": 0.2,
    "CODE_DIVERGENCE": 0.45,
    "OVERCODING": 0.5,
    "UNSUPPORTED_CODE": 0.6,
}

def _compute_risk(state: CodingState) -> tuple[float, str]:
    confidence = float(state.get("confidence_score", 0.5))
    discrepancy = state.get("discrepancy_type", "NO_COMPARISON")
    delta = abs(state.get("financial_delta") or 0.0)
    drg_flag = state.get("drg_flag")

    base_risk = round(1.0 - confidence, 4)  # Low confidence = high
    risk
    discrepancy_boost = DISCREPANCY_RISK.get(discrepancy, 0.2)
    delta_boost = min(delta / 5000.0, 0.2)  # Larger financial gaps =
    riskier

    if drg_flag == "MCC_MISSED":
        mcc_boost = 0.20
    elif drg_flag in ("CC_MISSED", "MCC_OVERCODED"):
        mcc_boost = 0.15

    raw_score = base_risk * 0.4 + discrepancy_boost * 0.4 + delta_boost * 0.1 +
    mcc_boost * 0.1
    score = round(min(raw_score, 1.0), 4)
    label = "LOW" if score < 0.35 else ("MEDIUM" if score <= 0.70 else "HIGH")
    return score, label
```

Explanation

After calculating the risk score, this node performs a multi-table atomic write to PostgreSQL: it inserts records into `clinical_cases`, `coding_results`, and `audit_log` within a single session, ensuring complete ACID compliance — either all three rows are written or none are.

NODE 9: Financial Calculator — `agents/financial_calculator.py`

Purpose: Apply the hospital-specific pricing multiplier to CPT codes and calculate the total estimated gross hospital revenue for the patient encounter.

```
@safe_node("financial_calc")
async def financial_calculator_node(state: CodingState) -> CodingState:
    cpt_codes = state.get("cpt_codes", [])

    # Fallback: if CPT resolver found nothing, use ICD base reimbursements
    directly.
    if not cpt_codes:
        icd_codes = state.get("icd_codes", [])
        icd_total = round(sum(float(c.get("base_reimbursement", 0)) for c in
icd_codes), 2)
        state["financial_summary"] = {
            "total_estimated_revenue": icd_total,
            "pricing_multiplier": 1.0,
            "line_items": []
        }
        return state

    # Fetch the hospital-specific multiplier from org_settings table
    org_id = state.get("org_id")
    multiplier = _get_org_multiplier(supabase, org_id) # Falls back to 1.0 on
failure

    # Apply multiplier to each CPT code to simulate the hospital's gross charge
    line_items = []
    total = 0.0
    for cpt in cpt_codes:
        base = float(cpt.get("base_price", 0.0))
        gross_charge = round(base * multiplier, 2)
        total += gross_charge
        line_items.append({
            "code": cpt.get("code"), "description": cpt.get("description"),
            "base_price": base, "multiplier": multiplier, "gross_charge":
gross_charge,
        })

    state["financial_summary"] = {
```

```

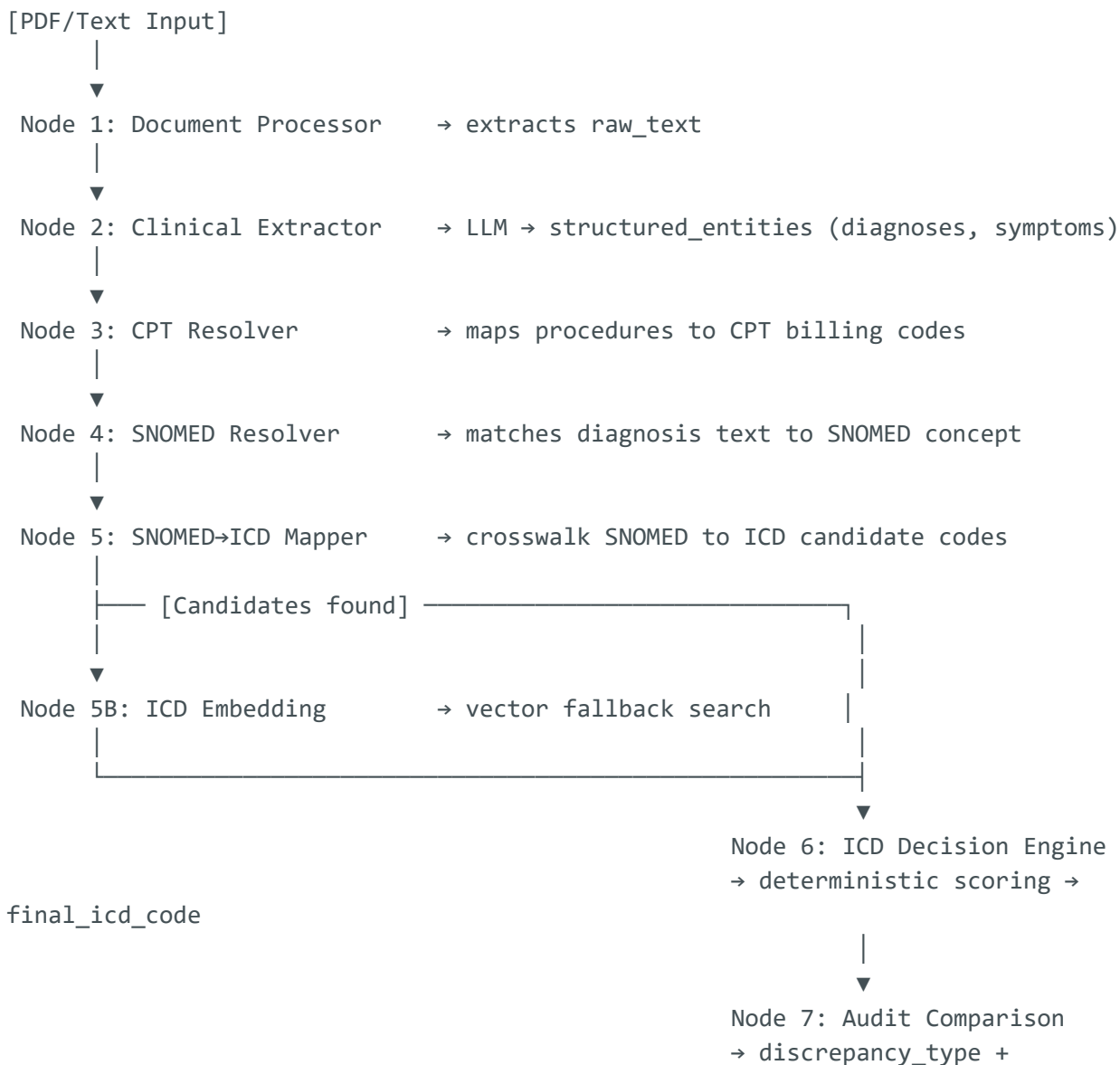
        "total_estimated_revenue": round(total, 2),
        "pricing_multiplier":      multiplier,
        "line_items":              line_items
    }
    return state

```

Explanation

The `cpt_pricing_multiplier` column in `org_settings` allows each hospital configured in the platform to customize their revenue calculation. A hospital with a multiplier of `1.5x` indicates their contracted rate is 50% above CMS Medicare base rates. This makes the financial output clinically realistic and per-hospital accurate.

Pipeline Flow Summary



financial_delta

|



Node 8: Risk Scorer
→ risk_score, risk_label, DB

write

|



Node 9: Financial Calculator
→ financial_summary

(total_estimated_revenue)

|



[END]